



UNIVERSIDAD PRIVADA DEL VALLE
FACULTAD DE INFORMÁTICA Y ELECTRÓNICA
INGENIERÍA DE SISTEMAS INFORMÁTICOS

Evaluación

PROGRAMACIÓN WEB II

Proyecto integrador

Manual técnico

Paralelo: “A”

Estudiantes:

- **Estevez Martinez Felipe Uriel**
- **Cussi Suxo Hugo Camilo**
- **Torrez Lopez Alan Steven**

Docente: Ing. Henry Miranda Ordonez

Gestión I - 2026

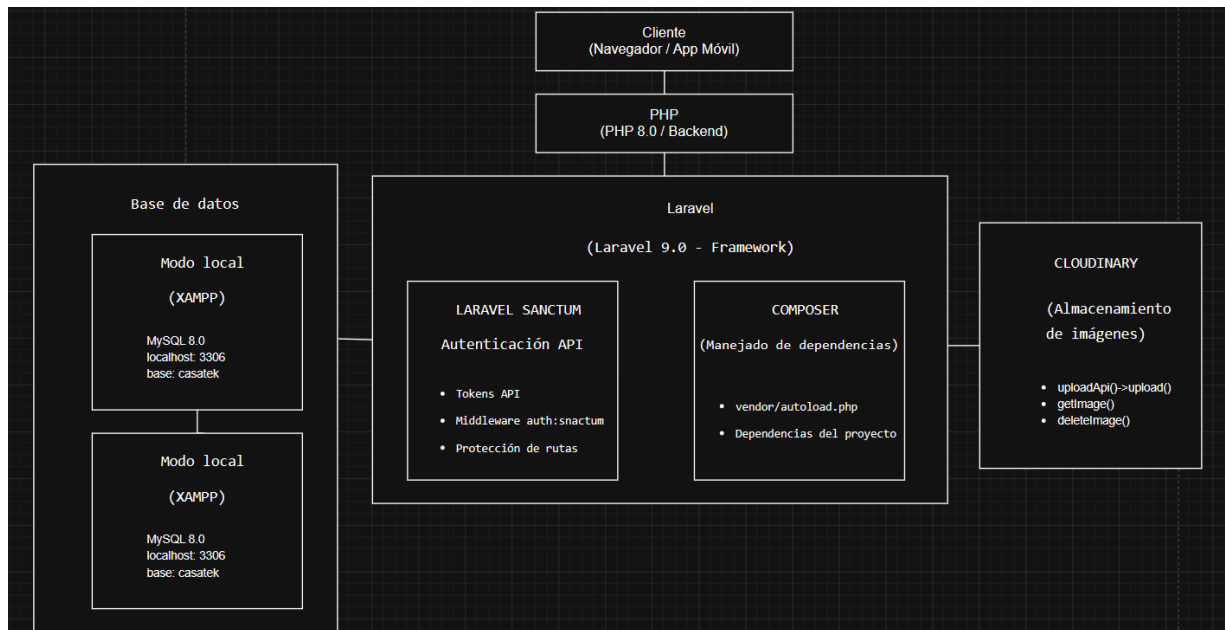
Contenido

1	Arquitectura del sistema	1
1.1	Diagrama de Arquitectura.....	1
2	Stack Tecnológico	1
2.1	Tecnologías y Herramientas:.....	1
2.2	Requisitos de Software.....	2
2.3	Extensiones PHP Requeridas	2
3	Variables de Entorno	4
3.1	Archivo .env Configuración.....	4
3.2	Configuración Especial para TiDB Cloud.....	6
3.3	Configuración en config/database.php (añadir para TiDB):	6
3.4	Alternar entre local y producción	7
4	Estructura de directorios clave.....	7
5	Base de datos.....	10
5.1	Modelo ER	10
5.2	Migraciones de la base de datos	11
5.2.1	Migraciones de tablas.....	11
5.2.2	Procedimientos almacenados y Triggers	17
6	Principales relaciones	23
7	APIs y Endpoints	26
7.1	Endpoints de la API.....	26
7.1.1	Rutas Públicas (sin autenticación).....	26

7.1.2	Rutas Protegidas (auth:sanctum)	26
7.1.3	Rutas de Administrador (auth:sanctum + middleware admin)	28
7.1.4	Documentación API (ejemplo con Thunder Client)	30
8	Middleware Implementados	44
9	Policies (Autorización)	45
10	Comandos Útiles	47

1 Arquitectura del sistema

1.1 Diagrama de Arquitectura



2 Stack Tecnológico

2.1 Tecnologías y Herramientas:

Componente	Tecnología	Versión
Backend	PHP	8
Framework	Laravel	9
Autenticación API	Laravel Sanctum	3.3
Debugging	Laravel Tinker	2.7
Base de Datos (Local)	MySQL (XAMPP)	8.0+
Base de Datos (Producción)	TiDB Cloud	8.5.3
Almacenamiento de Imágenes	Cloudinary	3.0.2
Servidor Local	XAMPP	3.3.0

2.2 Requisitos de Software

Requisito	Versión Mínima	Versión Actual Proyecto
PHP	>= 8.0	8.0
Composer	>= 2.0	2.0+
MySQL	>= 8.0	8.0+
Laravel	>= 9.0	9.0
Laravel Sanctum	>= 3.3	3.3

2.3 Extensiones PHP Requeridas

composer show --platform

composer 2.9.7 Composer package

composer-plugin-api 2.9.0 The Composer Plugin API

composer-runtime-api 2.2.2 The Composer Runtime API

ext-bcmath 8.2.12 The bcmath PHP extension

ext-calendar 8.2.12 The calendar PHP extension

ext-ctype 8.2.12 The ctype PHP extension

ext-curl 8.2.12 The curl PHP extension

ext-date 8.2.12 The date PHP extension

ext-fileinfo 8.2.12 The fileinfo PHP extension

ext-filter 8.2.12 The filter PHP extension

ext-gettext 8.2.12 The gettext PHP extension

ext-hash 8.2.12 The hash PHP extension

ext-iconv 8.2.12 The iconv PHP extension

ext-json 8.2.12 The json PHP extension

ext-libxml	8.2.12	The libxml PHP extension
ext-mysqli	8.2.12	The mysqli PHP extension
ext-mysqlnd	0	The mysqlnd PHP extension (actual version: mysqlnd 8.2.12)
ext-openssl	8.2.12	The openssl PHP extension
ext-pcre	8.2.12	The pcre PHP extension
ext-pdo_mysql	8.2.12	The pdo_mysql PHP extension
ext-pdo_sqlite	8.2.12	The pdo_sqlite PHP extension
ext-phar	8.2.12	The Phar PHP extension
ext-session	8.2.12	The session PHP extension
ext-spl	8.2.12	The SPL PHP extension
ext-tokenizer	8.2.12	The tokenizer PHP extension
ext-xml	8.2.12	The xml PHP extension
ext-xmlreader	8.2.12	The xmlreader PHP extension
ext-xmlwriter	8.2.12	The xmlwriter PHP extension
ext-zip	1.21.1	The zip PHP extension
ext-zlib	8.2.12	The zlib PHP extension
lib-bz2	1.0.8	The bz2 library
lib-curl	8.4.0	The curl library
lib-curl-libssh2	1.10.0	curl libssh2 version
lib-curl-openssl	3.0.11	curl OpenSSL version (3.0.11)

lib-curl-zlib 1.2.12 curl zlib version

lib-date-timelib 2022.09 date timelib version

lib-date-zoneinfo 2023.3 zoneinfo ("Olson") database for date

lib-mbstring-libmbfl 1.3.2 mbstring libmbfl version

lib-mbstring-oniguruma 6.9.8 mbstring oniguruma version

lib-openssl 3.0.11 OpenSSL 3.0.11 19 Sep 2023

lib-zip-libzip 1.7.1 The zip-libzip library

lib-zlib 1.2.12 The zlib library

php 8.2.12 The PHP interpreter

php-64bit 8.2.12 The PHP interpreter, 64bit

php-ipv6 8.2.12 The PHP interpreter, with IPv6 support

php-zts 8.2.12 The PHP interpreter, with Zend Thread Safety

3 Variables de Entorno

3.1 Archivo .env Configuración

CONFIGURACIÓN GENERAL DE LA APLICACIÓN

APP_NAME="CasaTek"

APP_ENV=local

APP_KEY=base64:kgiX00ZG1ltwA5g930hgzR#####

APP_URL=http://localhost

BASE DE DATOS - LOCAL (XAMPP)

DB_CONNECTION=mysql

DB_HOST=127.0.0.1

DB_PORT=3306

DB_DATABASE=casatek

BASE DE DATOS - PRODUCCIÓN (TiDB Cloud)

Descomentar para usar TiDB Cloud en producción

DB_CONNECTION=mysql

DB_HOST=gateway01.us-east-1.prod.aws.tidbcloud.com

DB_DATABASE=casatek

MYSQL_ATTR_SSL_CA=C:/certs/isrgrootx1.pem

MYSQL_ATTR_SSL_VERIFY_SERVER_CERT=false

CLOUDINARY - ALMACENAMIENTO DE IMÁGENES

CLOUDINARY_CLOUD_NAME=duzht###

CLOUDINARY_API_KEY=86387987#####

CLOUDINARY_API_SECRET=jsKlegVCRCK8xCS-#####

CLOUDINARY_URL=cloudinary://86387987#####:jsKlegVCRCK8xCS#####@duzh####

LARAVEL SANCTUM (Autenticación API)

SANCTUM_STATEFUL_DOMAINS=localhost

SESSION_DOMAIN=localhost

OTRAS CONFIGURACIONES

LOG_CHANNEL=stack

LOG_LEVEL=debug

BROADCAST_DRIVER=log

CACHE_DRIVER=file

QUEUE_CONNECTION=sync

SESSION_DRIVER=file

SESSION_LIFETIME=120

3.2 Configuración Especial para TiDB Cloud

Certificado SSL (para producción):

Ubicación del certificado

C:/certs/isrgrootx1.pem

3.3 Configuración en config/database.php (añadir para TiDB):

```
'mysql' => [  
    // ... config existente  
    'options' => [  
        PDO::MYSQL_ATTR_SSL_CA => env('MYSQL_ATTR_SSL_CA'),  
        PDO::MYSQL_ATTR_SSL_VERIFY_SERVER_CERT  
        env('MYSQL_ATTR_SSL_VERIFY_SERVER_CERT', false),  
    ],  
],
```

=>

3.4 Alternar entre local y producción

Para usar XAMPP local:

```
php artisan config:clear
```

Para cambiar a TiDB Cloud en producción:

```
# 1. Editar .env y descomentar bloque de TiDB Cloud
```

```
# 2. Comentar bloque de XAMPP
```

```
# 3. Limpiar cache de configuración
```

```
php artisan config:clear
```

```
php artisan config:cache
```

4 Estructura de directorios clave

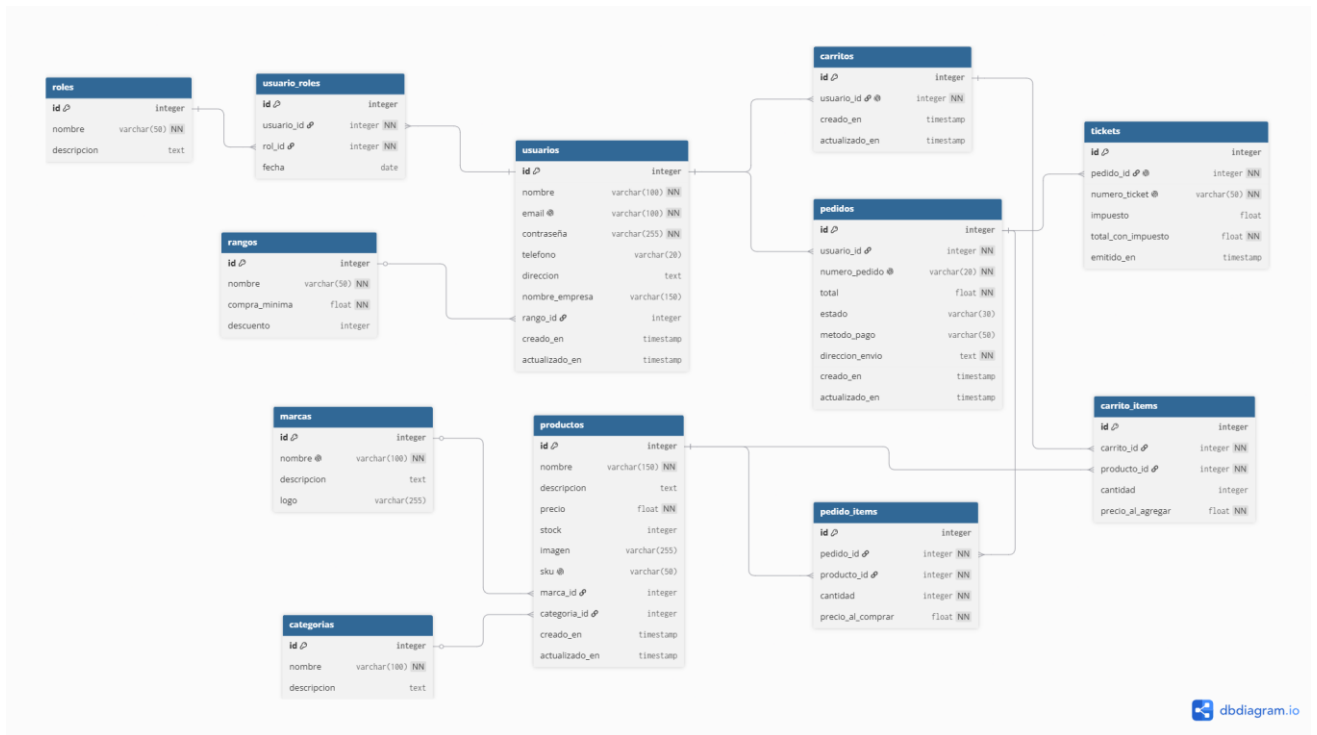
✓ app	Filtros de peticiones HTTP
> Console	Comandos Artisan personalizados
> Exceptions	Manejo global de excepciones
> Helpers	Funciones auxiliares personalizadas
✓ Http	
> Controllers	Controladores de la aplicación
> Middleware	Filtros de peticiones HTTP
Kernel.php	Registro de middleware y rutas
> Models	Modelos Eloquent (base de datos)
> policies	Políticas de autorización
> Providers	Service Providers de Laravel

>  app	Arranque del framework
>  bootstrap	Certificados SSL (personalizado)
>  certs	Configuraciones de Laravel
>  config	Base de datos
✓  database	Generadores de datos de prueba
>  factories	Estructura de tablas
>  migrations	Datos iniciales
>  seeders	
 .gitignore	
>  lang	Traducciones e idiomas
>  public	Acceso público (Document Root)
✓  resources	Recursos sin compilar
>  css	Archivos CSS fuente
>  js	Archivos JS fuente
✓  views	Plantillas Blade
>  admin	Vistas del panel administrativo
>  layouts	Plantillas base (header, footer)
>  partials	Componentes reutilizables

>  routes	Definición de rutas de la aplicación
>  storage	Archivos generados por Laravel
>  tests	Pruebas automatizadas
>  vendor	Dependencias de Composer
 .editorconfig	Configuración de formato de código
 .env	Variables de entorno (NO versionar)
 .env.example	Plantilla de variables de entorno
 .gitattributes	Configuración de atributos Git
 .gitignore	Archivos ignorados por Git
 .styleci.yml	Configuración de StyleCI (PHP code style)
 artisan	CLI de Laravel (ejecutable)
 composer.json	Dependencias de PHP
 composer.lock	Versiones exactas de dependencias
 package.json	Dependencias de Node.js
 phpunit.xml	Configuración de pruebas PHPUnit
 README.md	Documentación del proyecto
 webpack.mix.js	Compilador de assets (Laravel Mix)

5 Base de datos

5.1 Modelo ER



5.2 Migraciones de la base de datos

5.2.1 Migraciones de tablas

Las siguientes migraciones de Laravel conforman la estructura de la base de datos del sistema. Las migraciones se ejecutan en orden, según su nombre de archivo.

Archivo de migración	Función up()	Función down()
create_password_resets_table	Crea la tabla password_resets con los campos email (indexado), token y created_at. Se utiliza para el flujo nativo de recuperación de contraseña de Laravel.	Elimina la tabla password_resets con dropIfExists.
create_roles_table	Crea la tabla roles (id, name, description, timestamps) e inserta los tres roles del sistema: cliente, mayorista y admin. Los datos se insertan en la propia migración para garantizar que siempre existan.	Elimina la tabla roles con dropIfExists.
create_ranks_table	Crea la tabla ranks (id, name, monthly_minimum_purchase, description, timestamps) e inserta los cuatro niveles de fidelización: Bronce (0 Bs.),	Elimina la tabla ranks con dropIfExists.

Archivo de migración	Función up()	Función down()
	Plata (1 000 Bs.), Oro (5 000 Bs.) y Platino (10 000 Bs.).	
create_brands_table	Crea la tabla brands (id, name, description, logo, timestamps) para almacenar las marcas de los productos.	Elimina la tabla brands con dropIfExists.
create_categories_table	Crea la tabla categories (id, name, description, timestamps) para las categorías del catálogo.	Elimina la tabla categories con dropIfExists.
create_users_table	Crea la tabla users con los campos de autenticación de Laravel (name, email único, password, rememberToken) más campos propios: phone, whatsapp, access_code (para acceso mayorista) y rank_id (FK a ranks con nullOnDelete).	Elimina la tabla users con dropIfExists.
create_failed_jobs_table	Crea la tabla failed_jobs (id, uuid único, connection, queue, payload, exception, failed_at). Tabla estándar de Laravel para registrar trabajos de cola fallidos.	Elimina la tabla failed_jobs con dropIfExists.

Archivo de migración	Función up()	Función down()
create_personal_access_tokens_table	Crea la tabla personal_access_tokens con morphs tokenable, name, token (64 chars, único), abilities y last_used_at. Tabla estándar de Laravel Sanctum para autenticación por tokens.	Elimina la tabla personal_access_tokens con dropIfExists.
create_products_table	Crea la tabla products con name, description, base_price, stock, image, sku (único), warranty_days, brand_id (FK nullOnDelete) y category_id (FK nullOnDelete). Si se elimina la marca o categoría, el producto queda sin ella.	Elimina la tabla products con dropIfExists.
create_carts_table	Crea la tabla carts con user_id único y FK a users (cascade). La restricción unique garantiza que cada usuario tenga exactamente un carrito.	Elimina la tabla carts con dropIfExists.
create_cart_items_table	Crea la tabla cart_items con cart_id (FK cascade), product_id (FK cascade), quantity y price_when_added. El precio	Elimina la tabla cart_items con dropIfExists.

Archivo de migración	Función up()	Función down()
	se guarda en el momento de agregar para que no cambie si el producto se modifica después.	
create_billing_info_table	Crea la tabla billing_info con user_id (FK cascade), address, company_name, nit, business_name, whatsapp e is_default. Centraliza los datos de facturación/envío del usuario para reutilizarlos en pedidos.	Elimina la tabla billing_info con dropIfExists.
create_orders_table	Crea la tabla orders con user_id (FK cascade), billing_info_id (FK nullOnDelete), order_number único (formato ORD-XXXXXXXX), total, status (enum: pending, paid, shipped, delivered, cancelled) y payment_method.	Elimina la tabla orders con dropIfExists.
create_order_items_table	Crea la tabla order_items con order_id (FK cascade), product_id (FK cascade), quantity y price_when_ordered. Guarda el precio histórico de cada ítem para que no cambie si	Elimina la tabla order_items con dropIfExists.

Archivo de migración	Función up()	Función down()
	el producto es modificado después de la compra.	
create_tickets_table	Crea la tabla tickets con order_id único (FK cascade), ticket_number único e issued_at. La restricción unique en order_id garantiza que cada pedido tenga a lo sumo un ticket. El ticket se genera automáticamente vía trigger cuando el pedido pasa a delivered.	Elimina la tabla tickets con dropIfExists.
create_accumulated_purchases_table	Crea la tabla accumulated_purchases con user_id (FK cascade), month (fecha del primer día del mes), total_purchases y un índice único compuesto (user_id, month) para garantizar un solo registro por usuario por mes.	Elimina la tabla accumulated_purchases con dropIfExists.
create_user_role_table	Crea la tabla pivote user_role (user_id FK cascade, role_id FK cascade, assigned_at, timestamps) con un índice único compuesto (user_id, role_id) que impide asignar el mismo rol dos veces al	Elimina la tabla user_role con dropIfExists.

Archivo de migración	Función up()	Función down()
	mismo usuario.	
add_images_to_products_table	Agrega las columnas image1, image2, image3 e image4 (nullable) a la tabla products para admitir hasta cuatro imágenes por producto. Se realiza en migración separada porque el campo image original ya existía.	Elimina las columnas image1, image2, image3 e image4 de la tabla products.
add_expires_at_to_personal_access_tokens	Agrega la columna expires_at (timestamp nullable) a la tabla personal_access_tokens, después de la columna abilities. Requerida por versiones nuevas de Sanctum para soporte de expiración de tokens.	Elimina la columna expires_at de la tabla personal_access_tokens.
create_sessions_table	Crea la tabla sessions (id PK, user_id indexado, ip_address, user_agent, payload longtext, last_activity indexado). Tabla estándar de Laravel para almacenar sesiones en base de datos cuando el driver es database.	Elimina la tabla sessions con dropIfExists.

5.2.2 Procedimientos almacenados y Triggers

Esta migración registra todos los procedimientos almacenados y triggers del sistema mediante DB::unprepared(). La función up() los crea y la función down() los elimina en orden inverso.

5.2.2.1 Procedimientos Almacenados (up)

Nombre	Parámetros	Descripción	Contexto de uso
sp_filtrar_productos	p_nombre, p_category_id, p_brand_id, p_orden, p_per_page, p_offset	Filtra el catálogo público de productos por nombre (LIKE), categoría, marca y criterio de ordenamiento. Soporta paginación mediante p_per_page y p_offset. Retorna id, name, description, base_price, stock, sku, imágenes, warranty_days, brand_name y category_name.	Catálogo público del cliente.
sp_admin_filtrar_productos	p_search, p_category_id, p_brand_id, p_stock_min, p_stock_max, p_orden, p_per_page,	Versión extendida del filtro de productos para el panel de administración. Añade búsqueda en description y sku, además de filtros de	Panel de administración de productos.

Nombre	Parámetros	Descripción	Contexto de uso
	p_offset	stock mínimo y máximo. El valor por defecto de p_per_page es 15.	
sp_admin_filtrar_pedidos	p_search, p_status, p_user_id, p_from_date, p_to_date, p_per_page, p_offset	Filtra pedidos en el panel de administración por número de pedido, nombre o email del usuario, estado, usuario específico y rango de fechas. Retorna datos del pedido junto con información del usuario e items_count (subconsulta).	Panel de administración de pedidos.
sp_admin_filtrar_usuarios	p_search, p_role, p_rank_id, p_per_page, p_offset	Filtra usuarios por nombre, email o teléfono; también por rol (subconsulta EXISTS) y rango. Agrupa con GROUP_CONCAT para retornar todos los roles del usuario en un solo campo y calcula total_orders mediante	Panel de administración de usuarios.

Nombre	Parámetros	Descripción	Contexto de uso
		subconsulta.	
sp_admin_filtrar_tickets	p_search, p_from_date, p_to_date, p_per_page, p_offset	Filtra tickets por número de ticket, número de pedido, nombre o email del cliente, y rango de fechas de emisión. Retorna datos del ticket junto con información del pedido y del usuario.	Panel de administración de tickets.
sp_sales_report	p_start_date, p_end_date, p_status	Genera un reporte de ventas agrupado por día dentro de un rango de fechas. Calcula total de pedidos, ingresos totales, valor promedio por pedido y clientes únicos. Por defecto filtra pedidos con status = 'delivered'.	Reporte de ventas del panel de administración.
sp_top_products	p_limit, p_start_date, p_end_date	Retorna los productos más vendidos (por cantidad de unidades) en un rango de fechas.	Reporte de top productos del panel de administración.

Nombre	Parámetros	Descripción	Contexto de uso
		Calcula total_sold y total_revenue por producto, filtrando solo pedidos delivered. El límite por defecto es 10.	

5.2.2.2 Triggers (up)

Adicionalmente, la función up() crea la tabla auxiliar order_status_logs (id, order_id, old_status, new_status, changed_at) requerida por el trigger de log de estados.

Nombre	Evento	Momento	Descripción
trg_log_order_status	AFTER UPDATE ON orders	Posterior	Si el campo status cambia, inserta un registro en order_status_logs con el estado anterior, el nuevo y la fecha del cambio. Permite auditar el historial completo de estados de cada pedido.
trg_reduce_stock_on_order_item	AFTER INSERT ON order_items	Posterior	Al insertar un ítem en un pedido, descuenta la cantidad correspondiente del stock del producto. Solo aplica si el stock es

Nombre	Evento	Momento	Descripción
			suficiente (stock >= NEW.quantity).
trg_restore_stock_on_cancel	AFTER UPDATE ON orders	Posterior	Cuando un pedido cambia a estado cancelled (y no estaba ya cancelado), devuelve el stock de todos sus ítems a los productos correspondientes mediante un UPDATE con JOIN.
trg_auto_ticket_on_delivered	AFTER UPDATE ON orders	Posterior	Cuando un pedido pasa a delivered (y no estaba ya en ese estado), genera automáticamente un ticket con número correlativo en formato TKT-XXXXXXX, siempre que el pedido no tenga ya un ticket asociado.
trg_update_accumulated_purchases	AFTER UPDATE ON orders	Posterior	Al marcar un pedido como delivered, acumula el total del pedido en la tabla accumulated_purchases del usuario para el mes actual. Usa INSERT ...

- Elimina la tabla auxiliar order_status_logs.
- Elimina todos los stored procedures en orden inverso: sp_top_products, sp_sales_report, sp_admin_filtrar_tickets, sp_admin_filtrar_usuarios, sp_admin_filtrar_pedidos, sp_admin_filtrar_productos, sp_filtrar_productos.

6 Principales relaciones

User.php

- **belongsTo** Rank: cada usuario tiene un rango asignado (bronce, plata, oro, platino).
- **belongsToMany** Role (tabla pivot: user_role): un usuario puede tener varios roles simultáneamente; el pivot guarda assigned_at.
- **hasOne** Cart: cada usuario tiene un único carrito permanente.
- **hasMany** Order: un usuario puede tener múltiples pedidos.
- **hasMany** BillingInfo: un usuario puede guardar varias direcciones de facturación.
- **hasOne** BillingInfo (filtrado por is_default = true): acceso rápido a la dirección predeterminada.
- **hasMany** AccumulatedPurchase: historial de compras acumuladas mes a mes.

Order.php

- **belongsTo** User: el pedido pertenece a un usuario.
- **belongsTo** BillingInfo: el pedido referencia la dirección de facturación usada al momento de comprar.
- **hasMany** OrderItem: un pedido tiene múltiples ítems.
- **hasOne** Ticket: un pedido puede generar un comprobante (se crea cuando el estado llega a delivered).

Product.php

- **belongsTo** Brand: el producto pertenece a una marca.
- **belongsTo** Category: el producto pertenece a una categoría.
- **hasMany** CartItem: el producto puede estar en múltiples carritos.
- **hasMany** OrderItem: el producto puede aparecer en múltiples pedidos.

Cart.php

- **belongsTo** User: el carrito pertenece a un usuario.
- **hasMany** CartItem: el carrito contiene múltiples ítems.

CartItem.php

- **belongsTo** Cart: el ítem pertenece a un carrito.
- **belongsTo** Product: el ítem referencia un producto; guarda price_when_added para congelar el precio.

OrderItem.php

- **belongsTo** Order: el ítem pertenece a un pedido.
- **belongsTo** Product: el ítem referencia un producto; guarda price_when_ordered para congelar el precio.

BillingInfo.php

- **belongsTo** User: la dirección pertenece a un usuario
- **hasMany** Order: los pedidos que usaron esta dirección de facturación

Ticket.php

- **belongsTo** Order: el comprobante pertenece a un pedido

AccumulatedPurchase.php

- **belongsTo** User: el registro de compra acumulada pertenece a un usuario

Brand.php

- **hasMany** Product: una marca tiene múltiples productos

Category.php

- **hasMany** Product: una categoría tiene múltiples productos

Rank.php

- **hasMany** User: un rango puede tener muchos usuarios asignados

Role.php

- **belongsToMany** User (tabla pivot: user_role): un rol puede asignarse a muchos usuarios

7 APIs y Endpoints

7.1 Endpoints de la API

7.1.1 Rutas Públicas (sin autenticación)

Método	Endpoint	Controller@method	Descripción
POST	/api/register	AuthController@register	Registrar nuevo usuario
POST	/api/login	AuthController@login	Iniciar sesión
GET	/api/products	ProductController@index	Listar productos
GET	/api/products/search	ProductController@search	Buscar productos
GET	/api/products/{product}	ProductController@show	Ver detalle de producto
GET	/api/ranks	RankController@index	Listar rangos
GET	/api/ranks/{rank}	RankController@show	Ver detalle de rango

7.1.2 Rutas Protegidas (auth:sanctum)

Método	Endpoint	Controller@method	Descripción
POST	/api/logout	AuthController@logout	Cerrar sesión
GET	/api/me	AuthController@user	Ver perfil propio
PUT	/api/me/update	AuthController@updateProfile	Actualizar perfil propio

Método	Endpoint	Controller@method	Descripción
POST	/api/products	ProductController@store	Crear producto (solo admin)
PUT	/api/products/{product}	ProductController@update	Editar producto (solo admin)
DELETE	/api/products/{product}	ProductController@destroy	Eliminar producto (solo admin)
GET	/api/cart	CartController@index	Ver carrito
POST	/api/cart/add	CartController@add	Agregar ítem al carrito
PUT	/api/cart/items/{itemId}	CartController@update	Actualizar cantidad de ítem
DELETE	/api/cart/items/{itemId}	CartController@remove	Eliminar ítem del carrito
DELETE	/api/cart/clear	CartController@clear	Vaciar carrito
GET	/api/orders	OrderController@index	Listar pedidos del usuario
POST	/api/orders	OrderController@store	Crear pedido
GET	/api/orders/{order}	OrderController@show	Ver detalle de pedido
PATCH	/api/orders/{order}/cancel	OrderController@cancel	Cancelar pedido
GET	/api/orders/{order}/ticket	TicketController@showByOrder	Ver ticket de un pedido
GET	/api/billing-info	BillingInfoController@index	Listar direcciones de facturación

Método	Endpoint	Controller@method	Descripción
POST	/api/billing-info	BillingInfoController@store	Crear dirección de facturación
GET	/api/billing-info/{billingInfo}	BillingInfoController@show	Ver dirección de facturación
PUT	/api/billing-info/{billingInfo}	BillingInfoController@update	Editar dirección de facturación
PATCH	/api/billing-info/{billingInfo}/default	BillingInfoController@setDefault	Marcar como predeterminada
DELETE	/api/billing-info/{billingInfo}	BillingInfoController@destroy	Eliminar dirección de facturación
GET	/api/tickets	TicketController@index	Listar tickets del usuario
GET	/api/tickets/{ticket}	TicketController@show	Ver detalle de ticket
POST	/api/ranks	RankController@store	Crear rango (solo admin)
PUT	/api/ranks/{rank}	RankController@update	Editar rango (solo admin)
DELETE	/api/ranks/{rank}	RankController@destroy	Eliminar rango (solo admin)

7.1.3 Rutas de Administrador (auth:sanctum + middleware admin)

Método	Endpoint	Controller@method	Descripción
GET	/api/admin/dashboard	AdminController@dashboard	Resumen del dashboard

Método	Endpoint	Controller@method	Descripción
GET	/api/admin/reports/sales	AdminController@salesReport	Reporte de ventas
GET	/api/admin/reports/sales/sp	AdminController@salesReportProcedure	Reporte de ventas (stored procedure)
GET	/api/admin/reports/top-products	AdminController@topProductsProcedure	Productos más vendidos
GET	/api/admin/reports/customers	AdminController@customerStatisticsProcedure	Estadísticas de clientes
GET	/api/admin/reports/inventory	AdminController@inventoryAlertsProcedure	Alertas de inventario
GET	/api/admin/reports/executive	AdminController@executiveDashboardProcedure	Dashboard ejecutivo
GET	/api/admin/users	AdminController@users	Listar todos los usuarios
POST	/api/admin/users	AdminController@createUser	Crear usuario
GET	/api/admin/users/{user}	AdminController@showUser	Ver detalle de usuario
PUT	/api/admin/users/{user}	AdminController@updateUser	Editar usuario
DELETE	/api/admin/users/{user}	AdminController@deleteUser	Eliminar usuario
GET	/api/admin/products	AdminController@products	Listar productos (vista admin)
GET	/api/admin/orders	AdminController@orders	Listar todos los pedidos
PATCH	/api/admin/orders/{order}/status	AdminController@updateOrderStatus	Cambiar estado de pedido

Método	Endpoint	Controller@method	Descripción
GET	/api/admin/tickets	TicketController@adminIndex	Listar todos los tickets
POST	/api/admin/orders/{order}/ticket	TicketController@generate	Generar ticket para un pedido

7.1.4 Documentación API (ejemplo con Thunder Client)

Base URL: http://localhost:8000/api · Content-Type: application/json

7.1.4.1 Autenticación:

POST /register

POST http://localhost:8000/api/register

Status: 201 Created Size: 744 Bytes Time: 168 ms

JSON Content

```

1 {
2   "name": "Megumi Pérez",
3   "email": "megumi@correo.com",
4   "password": "miPassword123",
5   "password_confirmation": "miPassword123"
6 }

```

Response

```

1 {
2   "access_token": "24|1ZqvArU9EB79pMUChIYUmDRvZsawPgg8E0WLEiC41142d82",
3   "token_type": "Bearer",
4   "user": {
5     "name": "Megumi Pérez",
6     "email": "megumi@correo.com",
7     "rank_id": 1,
8     "updated_at": "2026-06-09T20:06:39.000000Z",
9     "created_at": "2026-06-09T20:06:39.000000Z",
10    "id": 22,
11    "roles": [
12      {
13        "id": 1,
14        "name": "cliente",
15        "description": "Cliente final",
16        "created_at": "2026-06-07T21:39:46.000000Z",
17        "updated_at": "2026-06-07T21:39:46.000000Z",
18        "pivot": {
19          "user_id": 22,
20          "role_id": 1

```

POST /login

POST http://localhost:8000/api/login

Status: 200 OK Size: 817 Bytes Time: 10.84 s

JSON Content

```

1 {
2   "email": "megumi@correo.com",
3   "password": "miPassword123"
4 }

```

Response

```

1 {
2   "access_token": "25|zd1FNzMMzmFQz9ea0Ne15jfyDo9FOOV4MeojWEW53b90da9",
3   "token_type": "Bearer",
4   "user": {
5     "id": 22,
6     "name": "Megumi Pérez",
7     "email": "megumi@correo.com",
8     "email_verified_at": null,
9     "phone": null,
10    "whatsapp": null,
11    "access_code": null,
12    "rank_id": 1,
13    "created_at": "2026-06-09T20:06:39.000000Z",
14    "updated_at": "2026-06-09T20:06:39.000000Z",
15    "roles": [
16      {
17        "id": 1,
18        "name": "cliente",
19        "description": "Cliente final",
20        "created_at": "2026-06-07T21:39:46.000000Z",

```

GET /me

GET `http://localhost:8000/api/me` Send

Status: 200 OK Size: 717 Bytes Time: 215 ms

Query Headers 2 Auth 1 **Body 1** Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content

```
1 {
2   "email": "megumi@correo.com",
3   "password": "miPassword123"
4 }
```

Response Headers 9 Cookies Results Docs

```
1 {
2   "id": 22,
3   "name": "Megumi Pérez",
4   "email": "megumi@correo.com",
5   "email_verified_at": null,
6   "phone": null,
7   "whatsapp": null,
8   "access_code": null,
9   "rank_id": 1,
10  "created_at": "2026-06-09T20:06:39.000000Z",
11  "updated_at": "2026-06-09T20:06:39.000000Z",
12  "roles": [
13    {
14      "id": 1,
15      "name": "cliente",
16      "description": "Cliente final",
17      "created_at": "2026-06-07T21:39:46.000000Z",
18      "updated_at": "2026-06-07T21:39:46.000000Z",
19      "pivot": {
```

PUT /me/update

PUT `http://localhost:8000/api/me/update` Send

Status: 200 OK Size: 780 Bytes Time: 173 ms

Query Headers 2 Auth 1 **Body 1** Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content

```
1 {
2   "name": "yunyun Editado",
3   "email": "yunyun@correo.com",
4   "phone": "77712345",
5   "whatsapp": "77712345",
6   "password": "nuevaPass123",
7   "password_confirmation": "nuevaPass123"
8 }
```

Response Headers 9 Cookies Results Docs

```
1 {
2   "message": "Perfil actualizado correctamente",
3   "user": {
4     "id": 22,
5     "name": "yunyun Editado",
6     "email": "yunyun@correo.com",
7     "email_verified_at": null,
8     "phone": "77712345",
9     "whatsapp": "77712345",
10    "access_code": null,
11    "rank_id": 1,
12    "created_at": "2026-06-09T20:06:39.000000Z",
13    "updated_at": "2026-06-09T20:10:39.000000Z",
14    "roles": [
15      {
16        "id": 1,
17        "name": "cliente",
18        "description": "Cliente final",
19        "created_at": "2026-06-07T21:39:46.000000Z",
20        "updated_at": "2026-06-07T21:39:46.000000Z",
```

POST /logout

POST `http://localhost:8000/api/logout` Send

Status: 200 OK Size: 52 Bytes Time: 103 ms

Query Headers 2 Auth 1 **Body 1** Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content

```
1 {
2   "name": "yunyun Editado",
3   "email": "yunyun@correo.com",
4   "phone": "77712345",
5   "whatsapp": "77712345",
6   "password": "nuevaPass123",
7   "password_confirmation": "nuevaPass123"
8 }
```

Response Headers 9 Cookies Results Docs

```
1 {
2   "message": "Sesión cerrada satisfactoriamente"
3 }
```

7.1.4.2 Productos:

GET /products:

The screenshot shows a REST client interface with a GET request to `http://localhost:8000/api/products?page=1&per_page=8&nombre=Camara&category_id=2&brand_id=1&orden=precio_asc`. The query parameters are listed on the left, and the response is a JSON object on the right.

Query Parameters:

Parameter	Value
page	1
per_page	8
nombre	Camara
category_id	2
brand_id	1
orden	precio_asc
parameter	value

Response (Status: 200 OK, Size: 747 Bytes, Time: 1.84 s):

```
{
  "datos": {
    "current_page": 1,
    "data": [],
    "first_page_url": "http://localhost:8000/api/products?per_page=8&nombre=Camara&category_id=2&brand_id=1&orden=precio_asc&page=1",
    "from": null,
    "last_page": 1,
    "last_page_url": "http://localhost:8000/api/products?per_page=8&nombre=Camara&category_id=2&brand_id=1&orden=precio_asc&page=1",
    "links": [
      {
        "url": null,
        "label": "&laquo; Previous",
        "active": false
      },
      {
        "url": "http://localhost:8000/api/products?per_page=8&nombre=Camara&category_id=2&brand_id=1&orden=precio_asc&page=1",
        "label": "1",

```

GET /products{id}:

The screenshot shows a REST client interface with a GET request to `http://localhost:8000/api/products/1`. The authentication is set to Bearer, and the response is a detailed JSON object for a specific product.

Auth: Bearer Token

Token Prefix: Bearer

Response (Status: 200 OK, Size: 907 Bytes, Time: 105 ms):

```
{
  "datos": {
    "id": 1,
    "name": "CAMARA PT WIFI 4MP",
    "description": "CAMARA PT WIFI 4MP PARA EXTERIORES\\r\\n*RASTERO DE MOVIMIENTO *DETEC.\\r\\nPERSONAS *FULL COLOR *IP65 *AUDIO\\r\\nBIDIRECCIONAL *DIST. ILUM 10M *RANURA\\r\\nMICRO SD MAX 256GB",
    "base_price": "375.00",
    "stock": 24,
    "image": null,
    "sku": "SKU-G122",
    "warranty_days": 365,
    "brand_id": null,
    "category_id": null,
    "created_at": "2026-06-07T21:51:17.000000Z",
    "updated_at": "2026-06-07T21:59:00.000000Z",
    "image1": "https://res.cloudinary.com/duzht7zvr/image/upload/v1780869071/casatek/productos/SKU-G122/6a25e7cd1b1f9.jpg",
    "image2": "https://res.cloudinary.com/duzht7zvr/image/upload
```

7.1.4.3 Carrito:

GET /cart:

The screenshot shows a REST client interface. The request is a GET to `http://localhost:8000/api/cart` with a Bearer token `27[3Nk8dqccijfNjID]6ZAcmp2TNUCMGdmlQK5W96e7aca1`. The response is a 200 OK with 110 bytes and 128 ms. The response body is a JSON object:

```
1 {
2   "datos": {
3     "cart_id": 20,
4     "items": [],
5     "total_items": 0,
6     "total_price": 0,
7     "is_empty": true
8   },
9   "message": "Carrito actual"
10 }
```

POST /cart/add

The screenshot shows a REST client interface. The request is a POST to `http://localhost:8000/api/cart/add` with a JSON body:

```
1 {
2   "product_id": 5,
3   "quantity": 2
4 }
```

The response is a 201 Created with 1.19 KB and 101 ms. The response body is a JSON object:

```
1 {
2   "datos": {
3     "cart": {
4       "cart_id": 20,
5       "items": [
6         {
7           "id": 45,
8           "cart_id": 20,
9           "product_id": 5,
10          "quantity": 2,
11          "price_when_added": "190.00",
12          "created_at": "2026-06-09T20:19:40.000000Z",
13          "updated_at": "2026-06-09T20:19:40.000000Z",
14          "product": {
15            "id": 5,
16            "name": "CAMARA SMART PTZ 3MP",
17            "description": "CAMARA SMART PTZ 3MP PARA INTERIOR\r\n\r\n*DETECCION DE MOVIMIENTO *AUDIO\r\n\r\nBIDIRECCIONAL\r\n\r\n*ALMACENAMIENTO DE 128GB\r\n\r\nVISION NOCTURNA *CONEXION WIFI\r\n\r\nDE\r\n\r\n2.4GHz:"
```

PUT /cart/items/{itemId}:

The screenshot shows a REST client interface with a PUT request to `http://localhost:8000/api/cart/items/45`. The request body is a JSON object: `{ "quantity": 3 }`. The response status is `200 OK` with a size of `1.06 KB` and a time of `128 ms`. The response body is a detailed JSON object:

```
1 {
2   "datos": {
3     "cart_item": {
4       "id": 45,
5       "cart_id": 20,
6       "product_id": 5,
7       "quantity": 3,
8       "price_when_added": "190.00",
9       "created_at": "2026-06-09T20:19:40.000000Z",
10      "updated_at": "2026-06-09T20:20:42.000000Z",
11      "product": {
12        "id": 5,
13        "name": "CAMARA SMART PTZ 3MP",
14        "description": "CAMARA SMART PTZ 3MP PARA INTERIOR\r\n*DETECCION DE MOVIMIENTO *AUDIO\r\n*8DIRECCIONAL *ALMACENAMIENTO DE 128GB\r\n*VISION NOCTURNA *CONEXION WIFI DE\r\n2.4GHz.",
15        "base_price": "190.00",
16        "stock": 36,
17        "image": null,
18        "sku": "SKU-G104",
19        "quantity_available": 100
20      }
21     }
22   }
23 }
```

DELETE /cart/items/{itemId}:

The screenshot shows a REST client interface with a DELETE request to `http://localhost:8000/api/cart/items/45`. The request body is a JSON object: `{ "quantity": 3 }`. The response status is `200 OK` with a size of `44 Bytes` and a time of `100 ms`. The response body is a simple JSON object:

```
1 {
2   "message": "Producto eliminado del carrito"
3 }
```

DELETE /cart/clear:

The screenshot shows a REST client interface with a DELETE request to `http://localhost:8000/api/cart/clear`. The request body is a JSON object: `{ "quantity": 3 }`. The response status is `200 OK` with a size of `42 Bytes` and a time of `88 ms`. The response body is a simple JSON object:

```
1 {
2   "message": "Carrito vaciado exitosamente"
3 }
```

7.1.4.4 Ordenes:

POST /orders:

The screenshot shows a REST client interface with a POST request to `http://localhost:8000/api/orders`. The request body is a JSON object representing an order. The response is a 201 Created status with a JSON body containing order details.

Request:

```
POST http://localhost:8000/api/orders
```

Request Body (JSON):

```
{
  "payment_method": "transferencia",
  "items": [
    { "product_id": 3, "quantity": 1 }
  ],
  "billing": {
    "address": "Calle Potosí #123",
    "nit": "1234567",
    "business_name": "Mi Empresa SRL",
    "whatsapp": "77712345"
  }
}
```

Response:

```
{
  "datos": {
    "billing_info_id": 17,
    "order_number": "ORD-00000052",
    "total": 970,
    "status": "pending",
    "payment_method": "transferencia",
    "user_id": 23,
    "updated_at": "2026-06-09T20:22:20.000000Z",
    "created_at": "2026-06-09T20:22:20.000000Z",
    "id": 50,
    "items": [
      {
        "id": 145,
        "order_id": 50,
        "product_id": 3,
        "quantity": 1,
        "price_when_ordered": "970.00",
        "created_at": "2026-06-09T20:22:20.000000Z",
        "updated_at": "2026-06-09T20:22:20.000000Z",
        "product": "f"
      }
    ]
  }
}
```

GET /orders:

The screenshot shows a REST client interface with a GET request to `http://localhost:8000/api/orders`. The response is a 200 OK status with a JSON body containing a list of orders.

Request:

```
GET http://localhost:8000/api/orders
```

Response:

```
{
  "datos": {
    "current_page": 1,
    "data": [
      {
        "id": 50,
        "user_id": 23,
        "billing_info_id": 17,
        "order_number": "ORD-00000052",
        "total": "970.00",
        "status": "pending",
        "payment_method": "transferencia",
        "created_at": "2026-06-09T20:22:20.000000Z",
        "updated_at": "2026-06-09T20:22:20.000000Z",
        "items": [
          {
            "id": 145,
            "order_id": 50,
            "product_id": 3,
            "quantity": 1,
            "price_when_ordered": "970.00"
          }
        ]
      }
    ]
  }
}
```

GET /orders{id}:

The screenshot shows a REST client interface with a GET request to `http://localhost:8000/api/orders/50`. The request body is a JSON object. The response status is 200 OK, with a size of 1.53 KB and a time of 104 ms. The response body is a JSON object containing order details.

```
1 {
2   "payment_method": "transferencia",
3   "items": [
4     { "product_id": 3, "quantity": 1 }
5   ],
6   "billing": {
7     "address": "Calle Potosí #123",
8     "nit": "1234567",
9     "business_name": "Mi Empresa SRL",
10    "whatsapp": "77712345"
11  }
12 }
```

```
1 {
2   "datos": {
3     "id": 50,
4     "user_id": 23,
5     "billing_info_id": 17,
6     "order_number": "ORD-00000052",
7     "total": "970.00",
8     "status": "pending",
9     "payment_method": "transferencia",
10    "created_at": "2026-06-09T20:22:20.000000Z",
11    "updated_at": "2026-06-09T20:22:20.000000Z",
12    "items": [
13      {
14        "id": 145,
15        "order_id": 50,
16        "product_id": 3,
17        "quantity": 1,
18        "price_when_ordered": "970.00",
19        "created_at": "2026-06-09T20:22:20.000000Z",
20        "updated_at": "2026-06-09T20:22:20.000000Z",
21        "product": {
```

PATCH /orders{id}/cancel:

The screenshot shows a REST client interface with a PATCH request to `http://localhost:8000/api/orders/50/cancel`. The request body is a JSON object. The response status is 200 OK, with a size of 43 Bytes and a time of 141 ms. The response body is a JSON object containing a success message.

```
1 {
2   "payment_method": "transferencia",
3   "items": [
4     { "product_id": 3, "quantity": 1 }
5   ],
6   "billing": {
7     "address": "Calle Potosí #123",
8     "nit": "1234567",
9     "business_name": "Mi Empresa SRL",
10    "whatsapp": "77712345"
11  }
12 }
```

```
1 {
2   "message": "Pedido cancelado exitosamente"
3 }
```


7.1.4.5 Datos de facturación

GET /billing-info:

The screenshot shows a REST client interface with a GET request to `http://localhost:8000/api/billing-info`. The status is 200 OK, size is 303 Bytes, and time is 138 ms. The response is a JSON object with a `datos` array containing one billing record and a `message` field.

```
1 {
2   "datos": [
3     {
4       "id": 17,
5       "user_id": 23,
6       "address": "Calle Potosí #123",
7       "company_name": null,
8       "nit": "1234567",
9       "business_name": "Mi Empresa SRL",
10      "whatsapp": "77712345",
11      "is_default": false,
12      "created_at": "2026-06-09T20:22:20.000000Z",
13      "updated_at": "2026-06-09T20:22:20.000000Z"
14    }
15  ],
16  "message": "Datos de facturación"
17 }
```

POST /billing-info:

The screenshot shows a REST client interface with a POST request to `http://localhost:8000/api/billing-info`. The status is 201 Created, size is 323 Bytes, and time is 117 ms. The request body is a JSON object with billing details. The response is a JSON object with a `datos` array containing one created record and a `message` field.

```
1 {
2   "datos": {
3     "user_id": 23,
4     "address": "Av. Mariscal Santa Cruz #1234",
5     "company_name": "Mi Empresa",
6     "nit": "1234567",
7     "business_name": "Mi Empresa SRL",
8     "whatsapp": "77712345",
9     "is_default": true,
10    "updated_at": "2026-06-09T20:26:45.000000Z",
11    "created_at": "2026-06-09T20:26:45.000000Z",
12    "id": 18
13  },
14  "message": "Datos de facturación creados"
15 }
```


PATCH /billing-info/{id}/{default}:

The screenshot shows a REST client interface with the following details:

- Method:** PATCH
- URL:** http://localhost:8000/api/billing-info/19/default
- Status:** 200 OK
- Size:** 331 Bytes
- Time:** 108 ms
- Body:** A JSON object with the following structure:

```
1 {
2   "address": "Av. Mariscal Santa Cruz #1234",
3   "company_name": "Mi Empresa",
4   "nit": "1234567",
5   "business_name": "Mi Empresa SRL",
6   "whatsapp": "77712345",
7   "is_default": true
8 }
```
- Response:** A JSON object with the following structure:

```
1 {
2   "datos": {
3     "id": 19,
4     "user_id": 23,
5     "address": "Av. Mariscal Santa Cruz #1234",
6     "company_name": "Mi Empresa",
7     "nit": "1234567",
8     "business_name": "Mi Empresa SRL",
9     "whatsapp": "77712345",
10    "is_default": true,
11    "created_at": "2026-06-09T20:27:56.000000Z",
12    "updated_at": "2026-06-09T20:27:56.000000Z"
13  },
14  "message": "Dirección predeterminada actualizada"
15 }
```

DELETE /billing-info/{id}:

The screenshot shows a REST client interface with the following details:

- Method:** DELETE
- URL:** http://localhost:8000/api/billing-info/19
- Status:** 200 OK
- Size:** 38 Bytes
- Time:** 102 ms
- Body:** A JSON object with the following structure:

```
1 {
2   "address": "Av. Mariscal Santa Cruz #1234",
3   "company_name": "Mi Empresa",
4   "nit": "1234567",
5   "business_name": "Mi Empresa SRL",
6   "whatsapp": "77712345",
7   "is_default": true
8 }
```
- Response:** A JSON object with the following structure:

```
1 {
2   "message": "Dirección eliminada"
3 }
```

7.1.4.6 Panel Admin (requiere token de administrador)

POST /admin/login:

The screenshot shows a REST client interface with a POST request to `http://localhost:8000/api/login`. The request body is a JSON object containing email and password. The response is a 200 OK status with a JSON body containing an access token and user details.

```
POST http://localhost:8000/api/login Send
```

Query Headers 2 Auth 1 **Body 1** Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 {
2   "email": "hugo@gmail.com",
3   "password": "12345678"
4 }
5
```

Status: 200 OK Size: 1.09 KB Time: 163 ms

Response Headers 9 Cookies Results Docs

```
1 {
2   "access_token": "28|H993J4pHU0TjrpOL8UXVTTaVjztD1enkNlCr8vIx428f3db2",
3   "token_type": "Bearer",
4   "user": {
5     "id": 1,
6     "name": "Hugo Camilo Cussi Suxo",
7     "email": "hugo@gmail.com",
8     "email_verified_at": null,
9     "phone": null,
10    "whatsapp": null,
11    "access_code": null,
12    "rank_id": 1,
13    "created_at": "2026-06-07T21:41:27.000000Z",
14    "updated_at": "2026-06-07T21:41:27.000000Z",
15    "roles": [
16      {
17        "id": 1,
18        "name": "cliente",
19        "description": "Cliente final",
20        "created_at": "2026-06-07T21:41:27.000000Z",
21        "updated_at": "2026-06-07T21:41:27.000000Z"
22      }
23    ]
24  }
25 }
```

GET /admin/dashboard:

The screenshot shows a REST client interface with a GET request to `http://localhost:8000/api/admin/dashboard`. The response is a 200 OK status with a JSON body containing dashboard statistics.

```
GET http://localhost:8000/api/admin/dashboard Send
```

Query Headers 2 Auth 1 **Body 1** Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 {
2   "email": "hugo@gmail.com",
3   "password": "12345678"
4 }
5
```

Status: 200 OK Size: 617 Bytes Time: 155 ms

Response Headers 9 Cookies Results Docs

```
1 {
2   "datos": {
3     "users": {
4       "total": 23,
5       "wholesalers": 4,
6       "final_customers": 18
7     },
8     "products": {
9       "total": 21
10    },
11    "orders": {
12      "total": 50,
13      "pending": 11,
14      "completed": 10
15    },
16    "sales": {
17      "total": "37135.00",
18      "by_month": [
19        {
20          "month": 6,
```

GET /admin/users:

GET `http://localhost:8000/api/admin/users?search=juan&role=mayorista&page=1` Send

Status: 200 OK Size: 561 Bytes Time: 93 ms

Query Headers 2 Auth 1 Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 {
2   "email": "hugo@gmail.com",
3   "password": "12345678"
4 }
5
```

Response Headers 9 Cookies Results Docs

```
1 {
2   "datos": {
3     "current_page": 1,
4     "data": [],
5     "first_page_url": "http://localhost:8000/api/admin/users?page=1",
6     "from": null,
7     "last_page": 1,
8     "last_page_url": "http://localhost:8000/api/admin/users?page=1",
9     "links": [
10      {
11        "url": null,
12        "label": "&laquo; Previous",
13        "active": false
14      },
15      {
16        "url": "http://localhost:8000/api/admin/users?page=1",
17        "label": "1",
18        "active": true
19      },
20      {
21        "url": null
```

POST /admin/users:

POST `http://localhost:8000/api/admin/users` Send

Status: 201 Created Size: 1000 Bytes Time: 167 ms

Query Headers 2 Auth 1 Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 {
2   "name": "María López",
3   "email": "maria@correo.com",
4   "password": "password123",
5   "phone": "77712345",
6   "rank_id": 1,
7   "roles": [2],
8   "billing": {
9     "address": "Calle 21 #456",
10    "nit": "9876543"
11  }
12 }
```

Response Headers 9 Cookies Results Docs

```
1 {
2   "datos": {
3     "name": "María López",
4     "email": "maria@correo.com",
5     "phone": "77712345",
6     "whatsapp": null,
7     "access_code": null,
8     "rank_id": 1,
9     "updated_at": "2026-06-09T20:34:38.000000Z",
10    "created_at": "2026-06-09T20:34:38.000000Z",
11    "id": 24,
12    "roles": [
13      {
14        "id": 2,
15        "name": "mayorista",
16        "description": "Cliente mayorista",
17        "created_at": "2026-06-07T21:39:46.000000Z",
18        "updated_at": "2026-06-07T21:39:46.000000Z",
19        "pivot": {
20          "user_id": 24,
```

PUT /admin/users/{id}:

The screenshot shows a REST client interface with a PUT request to `http://localhost:8000/api/admin/users/22`. The request body is a JSON object. The response status is 200 OK, with a size of 1.09 KB and a time of 120 ms. The response body is a JSON object containing user details.

```
PUT http://localhost:8000/api/admin/users/22
```

JSON Content

```
1 {
2   "name": "Nuevoyunyun Nombre",
3   "email": "nuevoyunyun@correo.com",
4   "rank_id": 2,
5   "roles": [1, 3]
6 }
```

Response

```
1 {
2   "datos": {
3     "id": 22,
4     "name": "Nuevoyunyun Nombre",
5     "email": "nuevoyunyun@correo.com",
6     "email_verified_at": null,
7     "phone": "77712345",
8     "whatsapp": "77712345",
9     "access_code": null,
10    "rank_id": 2,
11    "created_at": "2026-06-09T20:06:39.000000Z",
12    "updated_at": "2026-06-09T20:37:07.000000Z",
13    "roles": [
14      {
15        "id": 1,
16        "name": "cliente",
17        "description": "Cliente final",
18        "created_at": "2026-06-07T21:39:46.000000Z",
19        "updated_at": "2026-06-07T21:39:46.000000Z",
20        "pivot": {
```

DELETE /admin/users/{id}:

The screenshot shows a REST client interface with a DELETE request to `http://localhost:8000/api/admin/users/22?force=true`. The response status is 200 OK, with a size of 31 Bytes and a time of 111 ms. The response body is a JSON object containing a message.

```
DELETE http://localhost:8000/api/admin/users/22?force=true
```

JSON Content

```
1 {
2   "name": "Nuevoyunyun Nombre",
3   "email": "nuevoyunyun@correo.com",
4   "rank_id": 2,
5   "roles": [1, 3]
6 }
```

Response

```
1 {
2   "message": "Usuario eliminado"
3 }
```

GET /admin/orders:

The screenshot shows a REST client interface with a GET request to `http://localhost:8000/api/admin/orders`. The request body is a JSON object with the following structure:

```
1 {
2   "name": "Nuevoyun Nombre",
3   "email": "nuevoyun@correo.com",
4   "rank_id": 2,
5   "roles": [1, 3]
6 }
```

The response status is 200 OK, with a size of 51.43 KB and a time of 117 ms. The response body is a JSON object with the following structure:

```
1 {
2   "datos": {
3     "current_page": 1,
4     "data": [
5       {
6         "id": 50,
7         "user_id": 23,
8         "billing_info_id": 17,
9         "order_number": "ORD-00000052",
10        "total": "970.00",
11        "status": "cancelled",
12        "payment_method": "transferencia",
13        "created_at": "2026-06-09T20:22:20.000000Z",
14        "updated_at": "2026-06-09T20:24:52.000000Z",
15        "user": {
16          "id": 23,
17          "name": "Jorge Pérez",
18          "email": "jorge@correo.com",
19          "email_verified_at": null,
20          "phone": null,
21          "whatsapp": null
22        }
23      }
24     ]
25   }
26 }
```

PATCH /admin/orders/{id}/status:

The screenshot shows a REST client interface with a PATCH request to `http://localhost:8000/api/admin/orders/48/status`. The request body is a JSON object with the following structure:

```
1 { "status": "paid" }
```

The response status is 200 OK, with a size of 260 Bytes and a time of 128 ms. The response body is a JSON object with the following structure:

```
1 {
2   "datos": {
3     "id": 48,
4     "user_id": 8,
5     "billing_info_id": 7,
6     "order_number": "ORD-00000050",
7     "total": "7545.00",
8     "status": "paid",
9     "payment_method": "deposito",
10    "created_at": "2026-06-08T00:59:26.000000Z",
11    "updated_at": "2026-06-09T20:41:18.000000Z"
12  },
13  "message": "Estado actualizado"
14 }
```

GET /admin/products:

The screenshot shows a REST client interface with a GET request to `http://localhost:8000/api/admin/products`. The response status is 200 OK, with a size of 13.98 KB and a time of 102 ms. The response body is a JSON array containing one product object.

```
1 {
2   "datos": {
3     "current_page": 1,
4     "data": [
5       {
6         "id": 21,
7         "name": "Camara",
8         "description": "Camara de seguridad potente",
9         "base_price": "500.00",
10        "stock": 15,
11        "image": null,
12        "sku": "SAM -456",
13        "warranty_days": 365,
14        "brand_id": 23,
15        "category_id": 1,
16        "created_at": "2026-06-08T13:05:59.000000Z",
17        "updated_at": "2026-06-09T15:18:45.000000Z",
18        "image1": "https://res.cloudinary.com/duzht7zvr/image/upload/v1780923956/casatek/productos/SAM%20-456/6a26be324300e.jpg",
19        "image2": "https://res.cloudinary.com/duzht7zvr/image/upload/v1780923958/casatek/productos/SAM%20-456/6a26be3610A01.png"
20      }
21     ]
22   }
23 }
```

7.1.4.7 Reportes (Admin)

GET /admin/reports/sales:

The screenshot shows a REST client interface with a GET request to `http://localhost:8000/api/admin/reports/sales`. The response status is 200 OK, with a size of 191 Bytes and a time of 105 ms. The response body is a JSON object containing sales data for the month of August.

```
1 {
2   "datos": {
3     "period": "month",
4     "data": [
5       {
6         "period_label": "08",
7         "period": 8,
8         "total_orders": 9,
9         "total_sales": "29590.00"
10      }
11     ],
12   "summary": {
13     "total_orders": 9,
14     "total_sales": 29590
15   }
16 },
17 "message": "Reporte de ventas"
18 }
```

8 Middleware Implementados

Middlewares personalizados registrados en la aplicación.

Alias / Registro	Clase	Descripcion
admin	AdminMiddleware	Verifica que el usuario este autenticado y que isAdmin() sea true. Si falla, aborta con HTTP 403 'Acceso denegado'. Usado en el grupo /api/admin.
isAdmin	IsAdmin	Variante JSON del control de admin: devuelve { message: 'Solo administradores...' } con HTTP 403 en lugar de abort(). Adecuado para respuestas API puras.
auth	Authenticate	Extiende el middleware base de Laravel. Si la petición no espera JSON, redirige a la ruta 'iniciarsesion'; si es API devuelve 401 automáticamente.
guest	RedirectIfAuthenticated	Si el usuario ya está autenticado, lo redirige al HOME definido en RouteServiceProvider. Protege rutas como /login o /register desde la web.
—	EncryptCookies	Middleware estandar de Laravel. Encripta las cookies de la aplicación.
—	TrimStrings	Middleware estandar de Laravel. Elimina espacios en blanco al inicio y fin de los strings del request.
—	VerifyCsrfToken	Middleware estandar de Laravel. Valida el token CSRF en peticiones de formulario.
—	TrustProxies	Middleware estandar de Laravel. Configura proxies de confianza para obtener IPs reales.
—	TrustHosts	Middleware estandar de Laravel. Restringe los hosts permitidos.

Alias / Registro	Clase	Descripción
—	PreventRequestsDuringMaintenance	Middleware estandar de Laravel. Bloquea requests cuando la app está en modo mantenimiento.

9 Políticas (Autorización)

Reglas de autorización por recurso. Cada método devuelve true/false para permitir o denegar la acción al usuario.

Policy	Método	Quien puede	Descripción
BillingInfoPolicy	viewAny	Usuario autenticado	Puede listar sus propias direcciones de facturación
	view	Dueno o Admin	Puede ver una dirección específica
	create	Usuario autenticado	Puede agregar nuevas direcciones
	update	Dueno o Admin	Puede editar una dirección existente
	setDefault	Solo el dueño	Puede marcar una dirección como predeterminada (admin no puede hacerlo por otro usuario)
	delete	Dueno o Admin	Puede eliminar una dirección
OrderPolicy	viewAny	Usuario autenticado	Puede ver su historial de pedidos
	view	Dueno o Admin	Puede ver un pedido específico

Policy	Metodo	Quien puede	Descripcion
	create	Usuario autenticado	Puede crear nuevos pedidos
	cancel	Dueno + isPending()	Solo el dueño y unicamente si el pedido está en estado pendiente
	updateStatus	Solo Admin	Puede cambiar el estado de cualquier pedido
	viewAll	Solo Admin	Puede ver todos los pedidos del sistema
RankPolicy	viewAny	Publico (?User)	Sin autenticacion. El parametro \$user es nullable
	view	Publico (?User)	Sin autenticacion. Cualquiera puede ver un rango especifico
	create	Solo Admin	Puede crear nuevos rangos
	update	Solo Admin	Puede editar un rango existente
	delete	Solo Admin	Puede eliminar un rango
TicketPolicy	viewAny	Usuario autenticado	Puede listar sus propios tickets
	view	Dueno del pedido o Admin	El ticket pertenece al dueño del Order asociado, o es admin
	viewAll	Solo Admin	Puede ver todos los tickets del sistema

Policy	Metodo	Quien puede	Descripción
	<code>generate</code>	Solo Admin	Puede generar un ticket manualmente para cualquier pedido

10 Comandos Útiles

Comandos Laravel + GitHub + Cloudinary

1. Levantar el proyecto

Inicia el proyecto en local. Normalmente abre en `http://127.0.0.1:8000`

```
php artisan serve
```

2. Ver cambios realizados

Muestra qué archivos fueron modificados y cuáles están listos para commit. Usarlo siempre antes de hacer un commit.

```
git status
```

3. Traer cambios del repositorio

Descarga los cambios que subieron los compañeros y actualiza el proyecto local. Recomendado hacerlo antes de empezar a trabajar.

```
git pull origin main
```

3b. Traer cambios (versión corta)

Hace lo mismo si ya estás conectado a la rama correcta.

```
git pull
```

4a. Agregar un archivo específico al commit

Prepara ese archivo para el commit.

```
git add resources/views/admin/create-product.blade.php
```

4b. Agregar todos los archivos

Agrega todos los cambios realizados.

```
git add .
```

5. Crear un commit

Guarda una versión del trabajo localmente.

```
git commit -m "Descripcion del cambio"
```

6. Subir cambios a GitHub

Envía los commits al repositorio remoto.

```
git push origin main
```

6b. Subir cambios (versión corta)

Hace lo mismo si ya estás conectado a la rama.

```
git push
```

7. Ver en qué rama estás

Muestra la rama actual del proyecto.

```
git branch
```

8. Ver repositorio conectado

Verifica a qué repositorio de GitHub está conectado el proyecto.

```
git remote -v
```

9. Clonar proyecto de GitHub

Descarga el proyecto por primera vez.

```
git clone https://github.com/equipo/casatek.git
```

10. Instalar dependencias Laravel

Instala todas las librerías del proyecto.

```
composer install
```

11. Ejecutar migraciones

Crea las tablas de la base de datos.

```
php artisan migrate
```

12. Reiniciar migraciones

Borra todas las tablas y las vuelve a crear. Solo para desarrollo.

```
php artisan migrate:fresh
```

13. Ejecutar Seeders

Inserta datos de prueba en la base de datos.

```
php artisan db:seed
```

14a. Limpiar caché general

Limpia toda la caché del proyecto Laravel.

```
php artisan optimize:clear
```

14b. Limpiar configuración

Limpia la caché de configuración.

```
php artisan config:clear
```

14c. Limpiar rutas

Limpia la caché de rutas.

```
php artisan route:clear
```

14d. Limpiar vistas

Limpia la caché de vistas Blade.

php artisan view:clear

15. Ver rutas del proyecto

Muestra todas las rutas registradas en el proyecto.

php artisan route:list

16. Crear controlador

Genera un nuevo archivo de controlador.

php artisan make:controller ProductController

17. Crear modelo

Genera un nuevo archivo de modelo.

php artisan make:model Product

18. Crear migración

Genera un nuevo archivo de migración.

php artisan make:migration create_products_table

19. Crear modelo con migración

Genera un modelo y su migración al mismo tiempo.

php artisan make:model Product -m

20a. Verificar si Cloudinary está instalado

Busca el paquete en las dependencias instaladas. Si aparece cloudinary-labs/cloudinary-laravel, está instalado.

composer show | findstr cloudinary

20b. Verificar Cloudinary (forma directa)

Muestra información del paquete si está instalado.

composer show cloudinary-labs/cloudinary-laravel

21. Instalar Cloudinary

Descarga e instala el paquete de Cloudinary para Laravel.

```
composer require cloudinary-labs/cloudinary-laravel
```

22. Verificar variables Cloudinary

Abrir el archivo .env y verificar que estén presentes: CLOUDINARY_URL, CLOUDINARY_CLOUD_NAME, CLOUDINARY_API_KEY, CLOUDINARY_API_SECRET

.env

23. Verificar versión de Laravel

Muestra la versión de Laravel instalada en el proyecto.

```
php artisan --version
```

24. Verificar versión de PHP

Muestra la versión de PHP instalada en el sistema.

```
php -v
```